

Final Project Report

MUST2431

Nikolas Varga

Professor Ronald B Smith

The goal of my project was to develop a Max MSP patch that combined aspects of granular synthesis and frequency domain processing to create a spectral granulator. In researching spectral processing audio plugins, I found a project by Michael Norris, Soundmagic Spectral, to be a model of spectral processing. Soundmagic Spectral is a collection of advanced spectral processing tools that showed me creative applications of spectral processing. I also investigated spectral reverb briefly, but under the time restrictions of the project I was unable to realize spectral reverb in Max MSP (Vickers, Wu, Krishnan, & Sadanandam, 2007). Throughout the project, I referenced previous class patches and Max object Help pages to extend the techniques learned previously.

Within my final project, the user can select between three samples and a user-editable buffer to process. The audio is first processed granularly, then spectrally.



Figure 1: Sample selector and selector for audio to be processed.

The grain speed is controlled by a slider or a float object for more precise adjustments. Changing the grain speed adjusts the speed at which bangs are sent to three random objects: two within the main patch and the other within the window abstraction, p window. The random objects within the main patch generate a start point for a grain and a speed at which to play the sample. Within the window abstraction, the random object generates a length for each grain window.

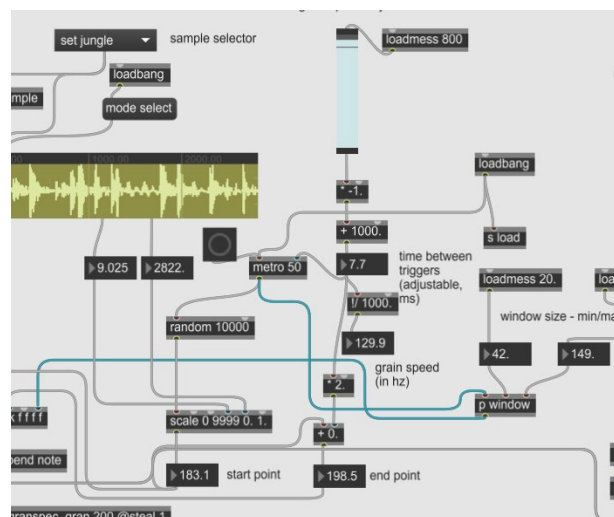


Figure 2: Start point, end point, grain speed, and window size control.

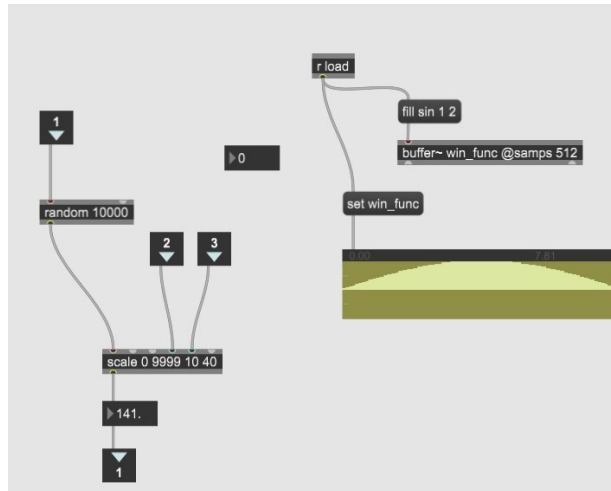


Figure 3: The window abstraction. The left side generates the window length, and the right side writes the window function to a buffer.

The grains are generated within the `poly~` object. The synthesis model follows that of polyphonic granular synthesis examples discussed in class, where a `groove~` object is used to access and play the sample, with `sig~` controlling the tuning of the grain. The `cycle~` object containing the window function is read through using the `line~` object, whose ramp length is determined in the window subpatch. With a random window length, grains have less of a chance of phase cancellation. The grain generator can be seen in Figure 4.

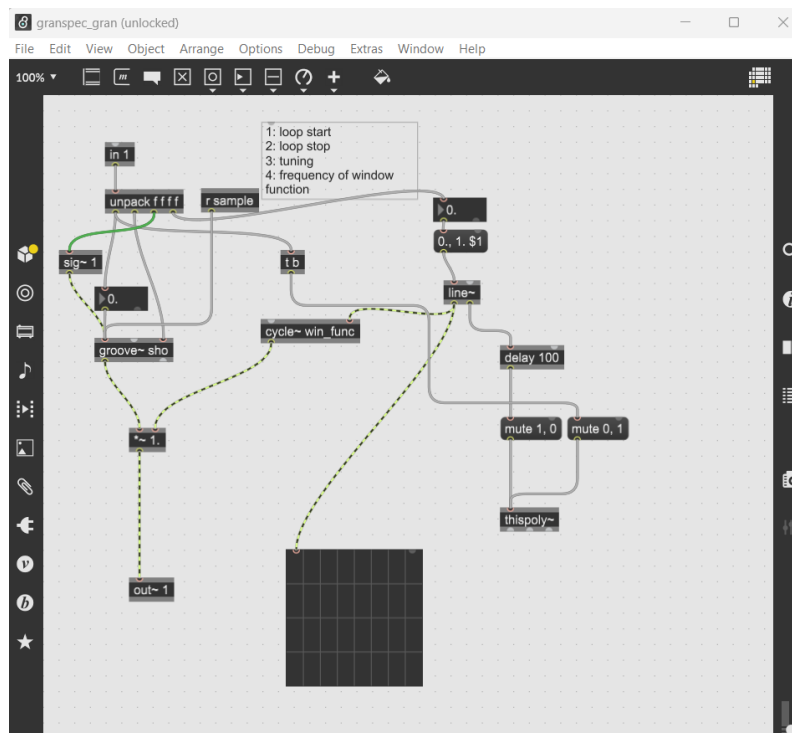


Figure 4: `granspec_gran` `poly~` sub-patch.

After the sample is granulated, the grains are passed into the spectral part of the patch. Figure 5 showcases the dry granular signal chain on the left, with effect dials in the middle of the patch. Below those is the pfft~ object, which controls the separation of grains into frequency bins and the delay implementation. Spectral effects are implemented both in the pfft~ subpatch and the main patch. In the pfft~ subpatch, the grains are separated into two to eight bins; the number of bins can be controlled by the user. Delay is applied to individual FFT samples within the pfft~ subpatch after they have been separated into bins, which can be seen in Figures 6 and 7. The index of an FFT sample is also used to determine the length of spectral delay and the amount of feedback for that sample. After being separated into bins and delayed, audio is routed back to the main patch, where distortion and reverb are applied to the bins. The parameters of the effects are controlled by multislider objects, as seen in Figure 9. Dry/wet mix is controlled by the subpatch in Figure 10 in the time domain and a variation of this in the frequency domain. The subpatch is controlled with a dial which ensures the signal can be fully dry, fully wet, or evenly mixed between both.

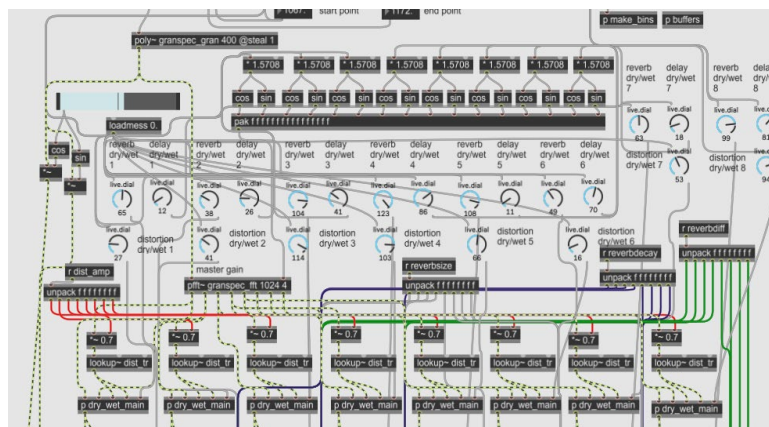


Figure 5: Dials, distortion objects, and dry/wet distortion control subpatches.

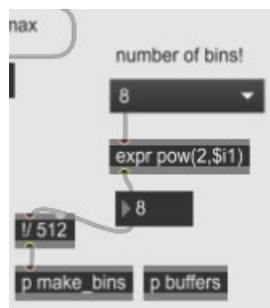
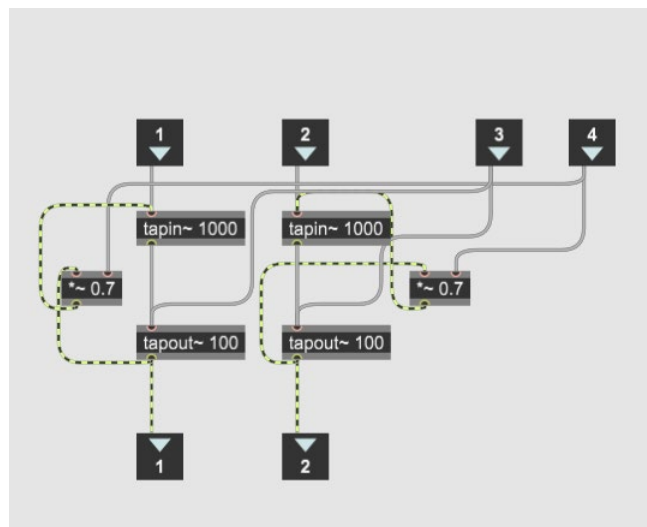
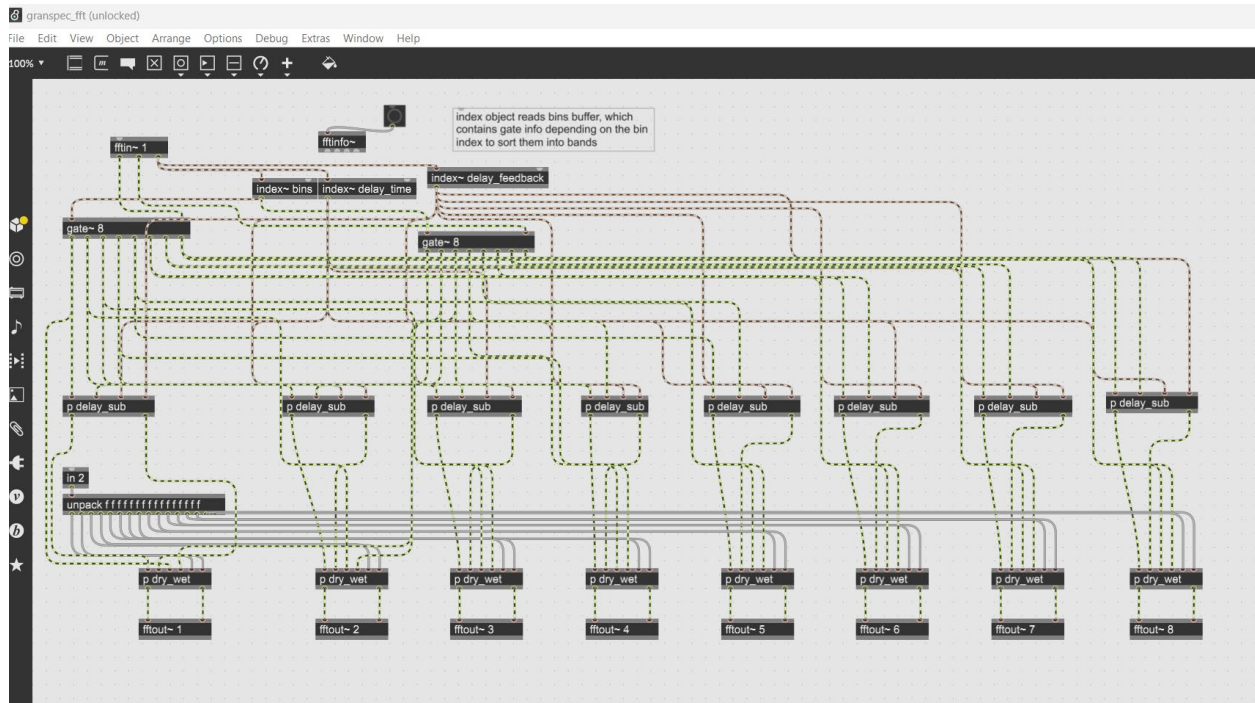


Figure 6: Bin chooser tool.



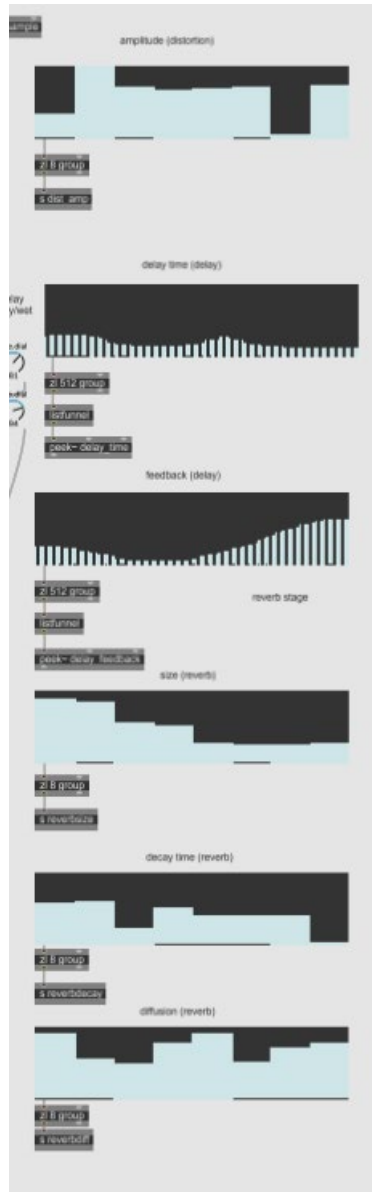


Figure 9: Multislider effect parameter controls. The delay parameters can be controlled more finely because the distortion and reverb are applied to bins rather than samples.

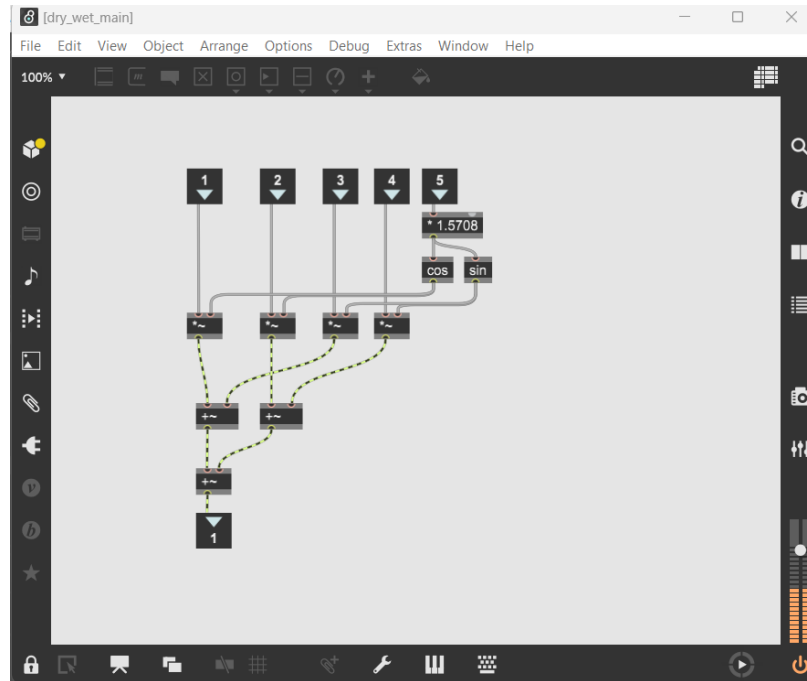


Figure 10: Dry/wet control subpatch. The left and right channels are individually mixed, then summed for a mono output.

I had difficulties throughout this project, but the most difficult part to implement was translating time domain effects into the frequency domain. I attempted to implement reverb and distortion in the frequency domain initially, trying to use both rectangular and polar coordinates for the effects. Ideally, all effects would have been able to be controlled sample-wise rather than bin-wise. While the distortion was more successful than the reverb, both ended up sounding better in the time domain. I suspect that the implementation of these effects in the frequency domain includes more specialized processing than their implementation in the time domain, which I was unable to complete by the project due date.

Another difficulty came in the form of controlling the effects. The first iteration of effect control was a umenu object selector with no control over the mix between dry and wet. After implementing reverb without dry/wet and delay with dry/wet, I realized that it would be easier to copy the dry/wet subpatch across all of the effects rather than mixing dry/wet control with umenus. Ultimately, this decision added more customizability to the patch.

The final patch used effects that could be applied to certain frequencies on a granular stream of audio. This fulfills my goal of combining spectral processing and granular processing. The delay effect accomplishes the goals of my project by applying typical time domain effects to the frequency domain of different grain streams. While the reverb and distortion do not accomplish the goal of applying an effect in the frequency domain, they still affect groups of frequencies of the granular audio differently.

In summary, this project represents a combination of frequency-based processing and granular processing of samples that can be used to create unique sounds. There are few examples of projects

that combine these two techniques, so this project represents a novel attempt in that field. Further steps to extend this project include using a custom reverb subpatch in the frequency domain to implement a spectral, sample-wise reverb and adding more bins for finer control over the output audio stream.

Works Cited

- Norris, M. (2023). *Soundmagic Spectral*. Retrieved from Michael Norris, Composer:
<https://michaelnorris.info/software/soundmagic-spectral>
- Smith, R. B. (2024, March 26). 03_26_24_Class_Patches. Boston, Massachusetts, United States of America.
- Smith, R. B. (2024, February 20). 2_20_24_Class_Patches. Boston, Massachusetts, United States of America.
- Vickers, E., Wu, J.-L., Krishnan, P., & Sadanandam, R. (2007). *Frequency Domain Artificial Reverberation*. Retrieved from
<https://nam12.safelinks.protection.outlook.com/?url=https%3A%2F%2Fwww.sfxmachine.com%2Fdocs%2FFDReverbSpectralMagDecay.pdf&data=05%7C02%7Cvarga.n%40northeastern.edu%7C19ddbd0d341a4061255008dc600698ee%7Ca8eec281aaa34daeac9b9a398b9215e7%7C0%7C0%7C6384908286>